

Networking Assignment Week 5

1. Try Test-Connection and nslookup commands for below websites: www.google.com
www.facebook.com www.amazon.com www.github.com www.cisco.com

```
PS C:\Users\HP> Test-Connection www.google.com -Count 4
```

Source	Destination	IPv4Address	IPv6Address	Bytes	Time(ms)
DESKTOP-MA...	www.google.com	142.251.155.119	2404:6800:4007:821::2004	32	44
DESKTOP-MA...	www.google.com	142.251.155.119	2404:6800:4007:821::2004	32	36
DESKTOP-MA...	www.google.com	142.251.155.119	2404:6800:4007:821::2004	32	43
DESKTOP-MA...	www.google.com	142.251.155.119	2404:6800:4007:821::2004	32	50

```
PS C:\Users\HP> Test-Connection www.facebook.com -Count 4
```

Source	Destination	IPv4Address	IPv6Address	Bytes	Time(ms)
DESKTOP-MA...	www.facebook...	57.144.242.1	2a03:2880:f334:1:face:b00c:0:25de	32	46
DESKTOP-MA...	www.facebook...	57.144.242.1	2a03:2880:f334:1:face:b00c:0:25de	32	50
DESKTOP-MA...	www.facebook...	57.144.242.1	2a03:2880:f334:1:face:b00c:0:25de	32	48
DESKTOP-MA...	www.facebook...	57.144.242.1	2a03:2880:f334:1:face:b00c:0:25de	32	39

```
PS C:\Users\HP> Test-Connection www.amazon.com -Count 4
```

Source	Destination	IPv4Address	IPv6Address	Bytes	Time(ms)
DESKTOP-MA...	www.amazon.com	18.161.240.42	2600:9000:257a:1400:7:49a5:5fd6:da1	32	45
DESKTOP-MA...	www.amazon.com	18.161.240.42	2600:9000:257a:1400:7:49a5:5fd6:da1	32	46
DESKTOP-MA...	www.amazon.com	18.161.240.42	2600:9000:257a:1400:7:49a5:5fd6:da1	32	51
DESKTOP-MA...	www.amazon.com	18.161.240.42	2600:9000:257a:1400:7:49a5:5fd6:da1	32	40

```
PS C:\Users\HP> Test-Connection www.github.com -Count 4
```

Source	Destination	IPv4Address	IPv6Address	Bytes	Time(ms)
DESKTOP-MA...	www.github.com	20.207.73.82		32	76
DESKTOP-MA...	www.github.com	20.207.73.82		32	71
DESKTOP-MA...	www.github.com	20.207.73.82		32	82
DESKTOP-MA...	www.github.com	20.207.73.82		32	67

```
PS C:\Users\HP> Test-Connection www.cisco.com -Count 4
```

Source	Destination	IPv4Address	IPv6Address	Bytes	Time(ms)
DESKTOP-MA...	www.cisco.com	23.41.120.121	2600:140f:5e00:480::b33	32	44
DESKTOP-MA...	www.cisco.com	23.41.120.121	2600:140f:5e00:480::b33	32	291
DESKTOP-MA...	www.cisco.com	23.41.120.121	2600:140f:5e00:480::b33	32	46
DESKTOP-MA...	www.cisco.com	23.41.120.121	2600:140f:5e00:480::b33	32	48

```
PS C:\Users\HP> nslookup www.google.com
Server:  dns.google
Address:  8.8.8.8

Non-authoritative answer:
Name:     www.google.com
Addresses: 2404:6800:4007:821::2004
          142.251.152.119
          142.251.157.119
          142.251.154.119
          142.251.156.119
          142.251.155.119
          142.251.151.119
          142.251.150.119
          142.251.153.119

PS C:\Users\HP> nslookup www.facebook.com
Server:  dns.google
Address:  8.8.8.8

Non-authoritative answer:
Name:     star-mini.c10r.facebook.com
Addresses: 2a03:2880:f334:1:face:b00c:0:25de
          57.144.54.1
Aliases:  www.facebook.com

PS C:\Users\HP> nslookup www.amazon.com
Server:  dns.google
Address:  8.8.8.8

Non-authoritative answer:
Name:     cf.47cf2c8c9-frontier.amazon.com
Addresses: 2600:9000:257a:fa00:7:49a5:5fd6:da1
          2600:9000:257a:8e00:7:49a5:5fd6:da1
          2600:9000:257a:0:7:49a5:5fd6:da1
          2600:9000:257a:4e00:7:49a5:5fd6:da1
          2600:9000:257a:7400:7:49a5:5fd6:da1
          2600:9000:257a:f000:7:49a5:5fd6:da1
          2600:9000:257a:5a00:7:49a5:5fd6:da1
          2600:9000:257a:8600:7:49a5:5fd6:da1
          108.158.60.199
Aliases:  www.amazon.com
          tp.47cf2c8c9-frontier.amazon.com
```

```
PS C:\Users\HP> nslookup www.github.com
Server:  dns.google
Address:  8.8.8.8

Non-authoritative answer:
Name:     github.com
Address:  20.207.73.82
Aliases:  www.github.com

PS C:\Users\HP> nslookup www.cisco.com
Server:  dns.google
Address:  8.8.8.8

Non-authoritative answer:
Name:     e2867.dsca.akamaiedge.net
Addresses: 2600:140f:6:1a7::b33
          2600:140f:6:18a::b33
          23.41.120.121
Aliases:  www.cisco.com
          www.cisco.com.akadns.net
          wwwds.cisco.com.edgekey.net
          wwwds.cisco.com.edgekey.net.globalredir.akadns.net
```

2) Use Wireshark to capture and analyze DNS, TCP, UDP traffic and packet header, packet flow, options and flags

- Wireshark was used to capture and analyze DNS, TCP, and UDP traffic.
- DNS packets were observed when resolving domain names into IP addresses. These packets use UDP protocol and contain query and response information.
- TCP packets were observed during data communication. TCP provides reliable transmission using sequence numbers, acknowledgements, and flags such as SYN, ACK, and FIN.
- UDP packets were also observed, which provide faster but connectionless communication.
- Each packet contains headers such as Ethernet header, IP header, and protocol-specific headers.
- The packet flow shows communication between the client and server through multiple layers.
- Flags in TCP packets indicate connection establishment and termination.
- This analysis demonstrates how different protocols work together for network communication.

dns						
No.	Time	Source	Destination	Protocol	Length	Info
4176...	3646.7825541...	192.168.1.5	192.168.1.1	DNS	132	Standard query 0xd75b F
4176...	3646.7843455...	192.168.1.1	192.168.1.5	DNS	211	Standard query response
4176...	3647.7275189...	192.168.1.5	192.168.1.1	DNS	132	Standard query 0xd928 F
4176...	3647.7297661...	192.168.1.1	192.168.1.5	DNS	211	Standard query response
4176...	3648.5925697...	192.168.1.5	192.168.1.1	DNS	80	Standard query 0x7733 A
4176...	3648.5925894...	192.168.1.5	192.168.1.1	DNS	80	Standard query 0x4b2c A
4176...	3648.6018315...	192.168.1.1	192.168.1.5	DNS	266	Standard query response
4176...	3648.6039057...	192.168.1.1	192.168.1.5	DNS	165	Standard query response
4177...	3648.7522024...	192.168.1.5	192.168.1.1	DNS	132	Standard query 0x4534 F
4177...	3648.7540564...	192.168.1.1	192.168.1.5	DNS	211	Standard query response
4177...	3649.6684869...	192.168.1.5	192.168.1.1	DNS	132	Standard query 0x2654 F
4177...	3649.6703512...	192.168.1.1	192.168.1.5	DNS	211	Standard query response

▶ Ethernet II, Src: DZS_c5:bc:cf (30:4f:75:c5:bc:cf), Dst: 20:58:ae:8e:bc:30	0000	20 58 43 e6 8e bc 30 4f 75
▶ Internet Protocol Version 4, Src: 192.168.1.1, Dst: 192.168.1.5	0010	00 70 8b af 40 00 40 11 2b
▶ User Datagram Protocol, Src Port: 53, Dst Port: 39713	0020	01 05 00 35 9b 21 00 5c c1
▼ Domain Name System (response)	0030	00 02 00 00 00 00 03 64 6e
Transaction ID: 0xf3aa	0040	65 00 00 1c 00 01 c0 0c 00
▶ Flags: 0x8180 Standard query response, No error	0050	00 10 20 01 48 60 48 60 00
Questions: 1	0060	88 88 c0 0c 00 1c 00 01 00
Answer RRs: 2	0070	48 60 48 60 00 00 00 00 00
Authority RRs: 0		
Additional RRs: 0		
▶ Queries		
▶ Answers		
[Request In: 110]		
[Time: 0.082850241 seconds]		

tcp						
No.	Time	Source	Destination	Protocol	Length	Info
4204...	3688.9102456...	2401:4900:1ce0:1424...	2404:6800:4007:839:...	TCP	86	43032 → 443 [ACK] Seq=7
4204...	3689.8760136...	192.168.1.5	217.146.11.106	TCP	90	54862 → 5938 [PSH, ACK]
4204...	3690.2342364...	192.168.1.5	217.146.11.106	TCP	90	[TCP Retransmission] 54
4204...	3690.2386139...	217.146.11.106	192.168.1.5	TCP	66	5938 → 54862 [ACK] Seq=
4204...	3690.3028301...	217.146.11.106	192.168.1.5	TCP	78	[TCP Dup ACK 420470#1]
4204...	3691.8763307...	192.168.1.5	217.146.11.106	TCP	90	54862 → 5938 [PSH, ACK]
4204...	3691.9615829...	217.146.11.106	192.168.1.5	TCP	66	5938 → 54862 [ACK] Seq=
4204...	3691.9615943...	217.146.11.106	192.168.1.5	TCP	90	5938 → 54862 [PSH, ACK]
4204...	3691.9616477...	192.168.1.5	217.146.11.106	TCP	66	54862 → 5938 [ACK] Seq=
4204...	3692.6722426...	192.168.1.5	217.146.11.106	TCP	122	54862 → 5938 [PSH, ACK]
4204...	3692.6868040...	217.146.11.106	192.168.1.5	TCP	122	5938 → 54862 [PSH, ACK]
4204...	3692.6868550...	192.168.1.5	217.146.11.106	TCP	66	54862 → 5938 [ACK] Seq=

Frame 420307: 86 bytes on wire (688 bits), 86 bytes captured (6...) Ethernet II, Src: 20:58:43:e6:8e:bc (20:58:43:e6:8e:bc), Dst: D... Internet Protocol Version 6, Src: 2401:4900:1ce0:1424:ee49:a47b... Transmission Control Protocol, Src Port: 44452, Dst Port: 443, Source Port: 44452 Destination Port: 443 [Stream index: 419] [Conversation completeness: Incomplete, DATA (15)] [TCP Segment Len: 0] Sequence Number: 330060 (relative sequence number) Sequence Number (raw): 2920824859 [Next Sequence Number: 330060 (relative sequence number)] Acknowledgment Number: 100018 (relative ack number) Acknowledgment number (raw): 957311290 1000... = Header Length: 22 bytes (0)	<pre> 0000 30 4f 75 c5 bc cf 20 58 43 0010 bc 14 00 20 06 40 24 01 48 0020 a4 7b 1e 6a a9 e7 26 06 47 0030 00 00 5f f2 75 c4 ad a4 01 0040 69 3a 80 10 09 24 ab 2f 0c 0050 e8 f2 85 70 00 80 </pre>
--	---

udp						
No.	Time	Source	Destination	Protocol	Length	Info
4210...	3717.6523244...	2401:4900:1ce0:1424...	2401:4900:1ce0:1424...	UDP	1454	52900 → 39770 Len=1392
4210...	3717.6523432...	2401:4900:1ce0:1424...	2401:4900:1ce0:1424...	UDP	1454	52900 → 39770 Len=1392
4210...	3717.6523575...	2401:4900:1ce0:1424...	2401:4900:1ce0:1424...	UDP	1454	52900 → 39770 Len=1392
4210...	3717.6523576...	2401:4900:1ce0:1424...	2401:4900:1ce0:1424...	UDP	1454	52900 → 39770 Len=1392
4210...	3717.6523576...	2401:4900:1ce0:1424...	2401:4900:1ce0:1424...	UDP	1454	52900 → 39770 Len=1392
4210...	3717.6523577...	2401:4900:1ce0:1424...	2401:4900:1ce0:1424...	UDP	78	52900 → 39770 Len=16
4210...	3717.6523578...	2401:4900:1ce0:1424...	2401:4900:1ce0:1424...	UDP	86	52900 → 39770 Len=24
4210...	3717.6530428...	2401:4900:1ce0:1424...	2401:4900:1ce0:1424...	UDP	110	39770 → 52900 Len=48
4210...	3717.6564015...	2401:4900:1ce0:1424...	2401:4900:1ce0:1424...	UDP	129	52900 → 39770 Len=67
4210...	3717.6649358...	2401:4900:1ce0:1424...	2401:4900:1ce0:1424...	UDP	86	39770 → 52900 Len=24
4210...	3717.8153068...	192.168.1.5	192.168.1.1	DNS	132	Standard query 0x36e6 F
4210...	3717.8172522...	192.168.1.1	192.168.1.5	DNS	211	Standard query response

Frame 420302: 161 bytes on wire (1288 bits), 161 bytes captured (1...) Ethernet II, Src: 20:58:43:e6:8e:bc (20:58:43:e6:8e:bc), Dst: DZS... Internet Protocol Version 6, Src: 2401:4900:1ce0:1424:ee49:a47b:1... User Datagram Protocol, Src Port: 35472, Dst Port: 443 Source Port: 35472 Destination Port: 443 Length: 107 Checksum: 0xd5a [unverified] [Checksum Status: Unverified] [Stream index: 430] [Timestamps] UDP payload (99 bytes) QUIC IETF	<pre> 0000 30 4f 75 c5 bc cf 20 58 43 0010 4a 9c 00 6b 11 40 24 01 48 0020 a4 7b 1e 6a a9 e7 24 04 68 0030 00 00 00 00 20 03 8a 90 00 0040 fa f0 ca de c7 c4 c3 30 90 0050 9b 5b b5 da 96 b7 50 6a 60 0060 d5 97 1c 8a 86 3d 8a 09 80 0070 c6 5b c8 2c 66 a8 20 9b 00 0080 1b 9c 9e bd 8b bc 2a fe 80 0090 ac 0a 39 4d ad ae 44 34 e0 00a0 59 </pre>
--	---

3.Explore traceroute/tracert for different websites eg:google.com and analyse the parameters in the output and explore different options for traceroute command

- The traceroute command was used to analyze the path taken by packets to reach www.google.com.
- The output shows multiple hops, where each hop represents a router between the source and destination.
- Each hop displays the IP address of the router and the round-trip time in milliseconds.
- The first hop corresponds to the local router (192.168.1.1), followed by intermediate ISP routers, and finally the destination server.
- Some hops displayed “* * *”, indicating that the router did not respond to traceroute requests.
- Different options of traceroute were explored:
- The -n option displays only IP addresses without resolving hostnames, making it faster.
- The -I option uses ICMP packets instead of UDP for tracing the route.
- The -m option limits the maximum number of hops, as shown with -m 5.
- The results show how packets travel through multiple routers across the network to reach the destination.

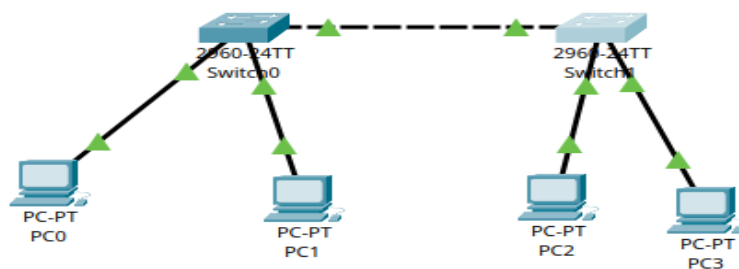
```
boss@boss:~$ traceroute www.google.com
traceroute to www.google.com (142.251.152.119), 30 hops max, 60 byte packets
 1 _gateway (10.246.151.73)  4.019 ms  3.993 ms  3.982 ms
 2 * * *
 3 255.0.0.2 (255.0.0.2)  38.872 ms  39.188 ms  40.177 ms
 4 255.0.0.3 (255.0.0.3)  37.831 ms  38.717 ms  38.696 ms
 5 192.168.225.99 (192.168.225.99)  37.802 ms  192.168.225.98 (192.168.225.98)  37.227 ms  37.217 ms
 6 192.168.83.26 (192.168.83.26)  37.196 ms  192.168.83.24 (192.168.83.24)  32.057 ms  192.168.83.28 (192.168.83.28)  32.091 ms
 7 * * *
 8 * * *
 9 142.251.152.119 (142.251.152.119)  37.803 ms  209.85.175.48 (209.85.175.48)  38.582 ms  142.251.152.119 (142.251.152.119)  37.733 ms
```

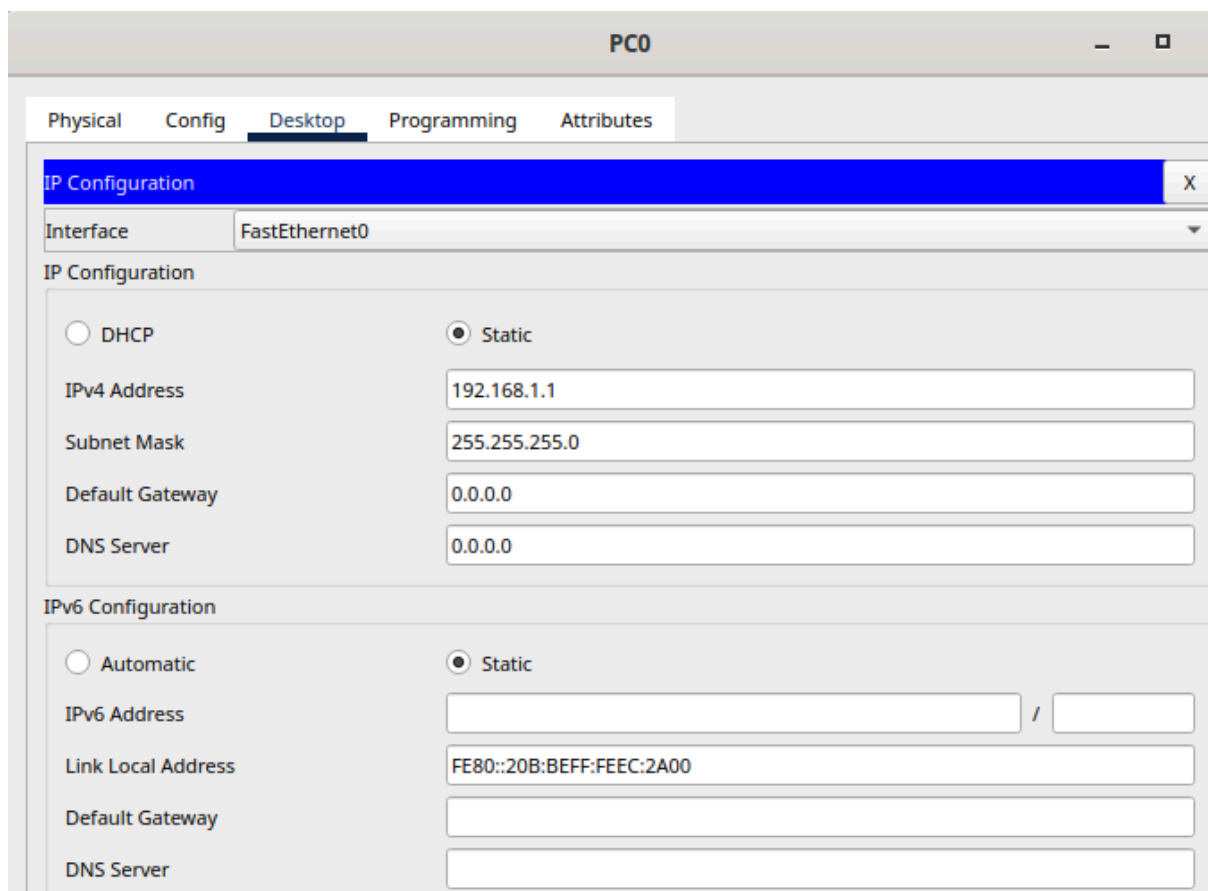
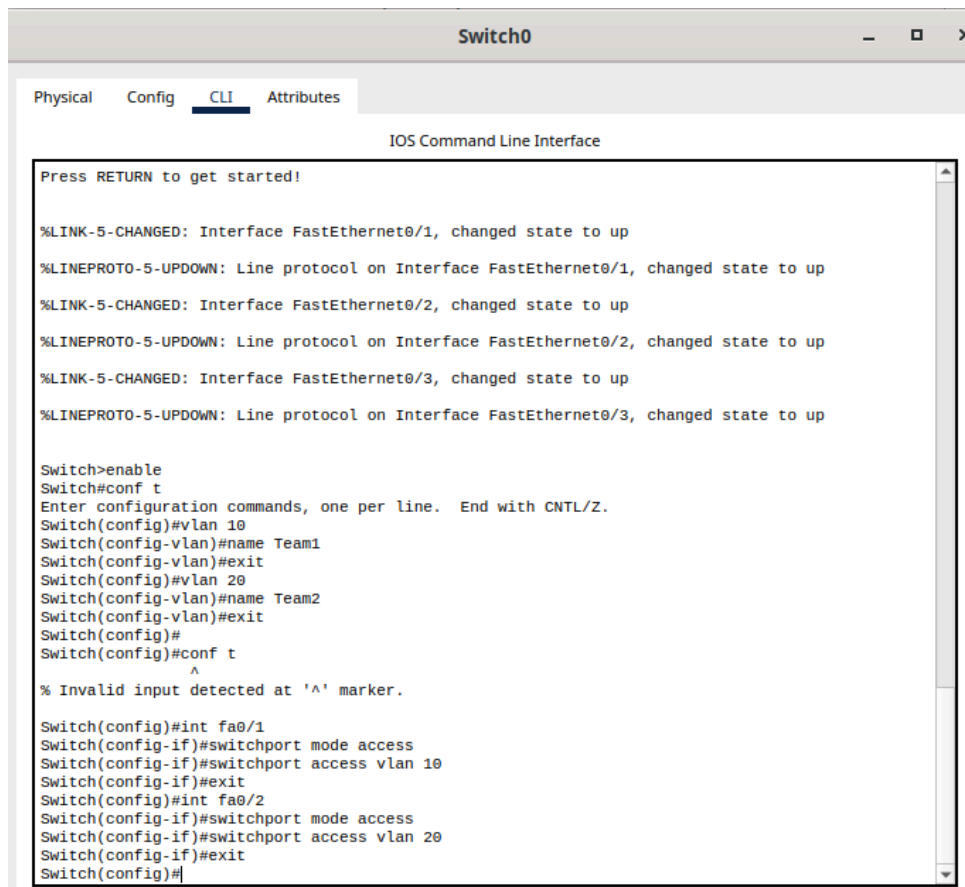
```
boss@boss:~$ traceroute -n www.google.com
traceroute to www.google.com (142.251.152.119), 30 hops max, 60 byte packets
 1  10.246.151.73  4.102 ms  4.042 ms  4.298 ms
 2  * * *
 3  255.0.0.2  205.016 ms  205.004 ms  205.300 ms
 4  255.0.0.3  204.951 ms  203.528 ms  204.898 ms
 5  192.168.225.98  206.449 ms  192.168.225.99  205.491 ms  192.168.225.98  206.430 ms
 6  192.168.83.28  205.052 ms  200.825 ms  201.435 ms
 7  * * *
 8  * * *
 9  209.85.175.48  202.671 ms  202.869 ms  203.173 ms
10  142.251.152.119  202.917 ms  202.293 ms  202.069 ms

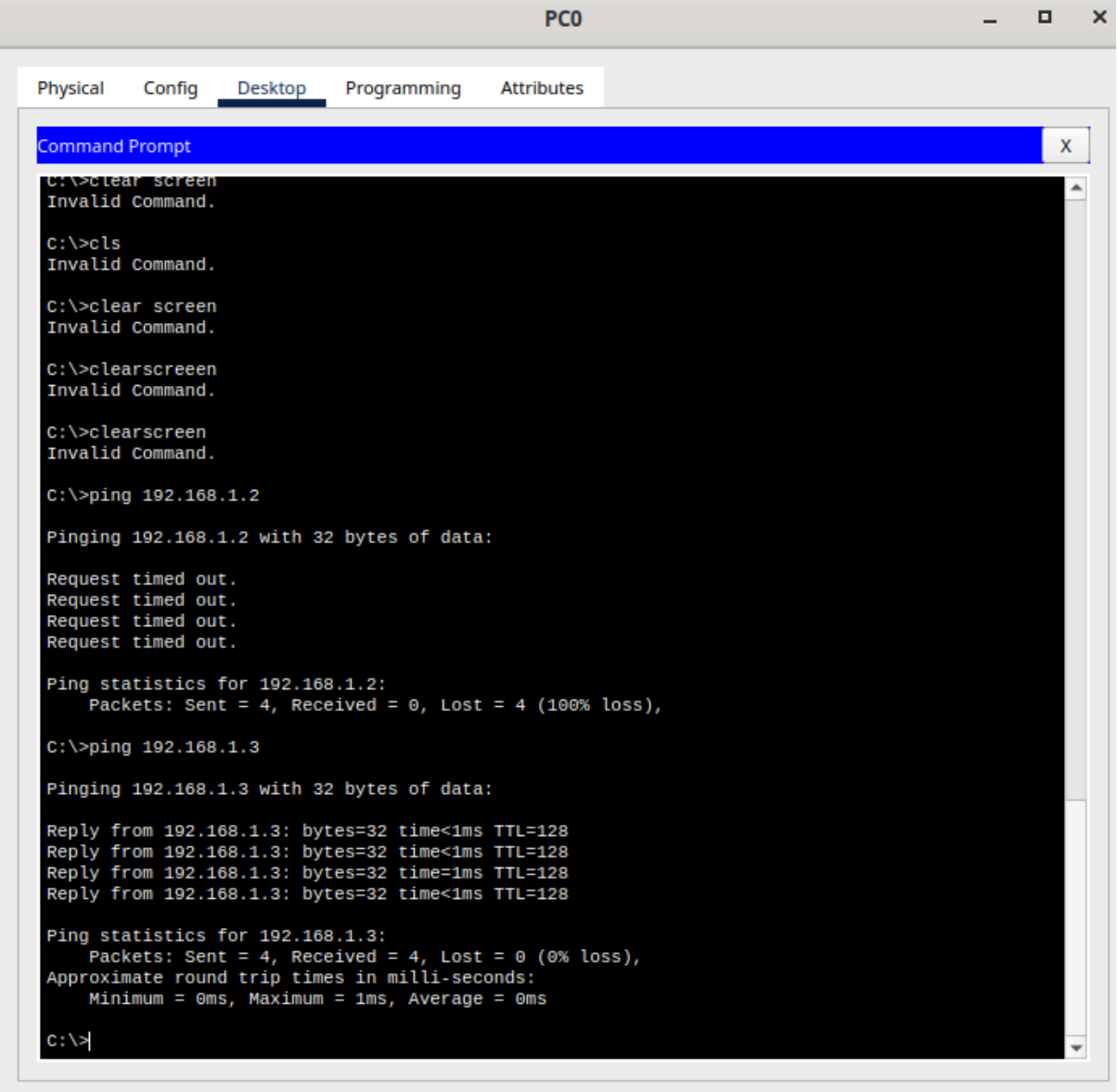
boss@boss:~$ sudo traceroute -I www.google.com
traceroute to www.google.com (142.250.183.164), 30 hops max, 60 byte packets
 1  _gateway (10.246.151.73)  3.923 ms  3.919 ms  4.118 ms
 2  * * *
 3  255.0.0.2 (255.0.0.2)  138.473 ms  139.587 ms  141.244 ms
 4  255.0.0.3 (255.0.0.3)  138.306 ms  138.663 ms  138.728 ms
 5  192.168.225.98 (192.168.225.98)  139.345 ms  140.023 ms  140.711 ms
 6  192.168.83.28 (192.168.83.28)  140.491 ms  150.925 ms  150.888 ms
 7  * * *
 8  * * *
 9  72.14.217.252 (72.14.217.252)  30.971 ms  30.728 ms  30.820 ms
10  216.239.43.131 (216.239.43.131)  28.739 ms  29.212 ms  35.703 ms
11  142.251.55.65 (142.251.55.65)  35.659 ms  27.212 ms  26.272 ms
12  lcmaaa-ar-in-f4.1e100.net (142.250.183.164)  26.594 ms  31.448 ms  30.878 ms

boss@boss:~$ traceroute -m 5 www.google.com
traceroute to www.google.com (142.251.152.119), 5 hops max, 60 byte packets
 1  _gateway (10.246.151.73)  20.412 ms  20.382 ms  20.511 ms
 2  * * *
 3  255.0.0.2 (255.0.0.2)  62.945 ms  63.275 ms  63.259 ms
 4  255.0.0.3 (255.0.0.3)  62.465 ms  62.444 ms  62.426 ms
 5  192.168.225.98 (192.168.225.98)  62.394 ms  192.168.225.99 (192.168.225.99)
```

5.Set up a trunk port between switches and try ping between different VLANs







The screenshot shows a PC0 window with tabs for Physical, Config, Desktop, Programming, and Attributes. The Desktop tab is active, displaying a Command Prompt window. The Command Prompt shows the following commands and their outputs:

```
C:\>clear screen
Invalid Command.

C:\>cls
Invalid Command.

C:\>clear screen
Invalid Command.

C:\>clearscreen
Invalid Command.

C:\>clearscreen
Invalid Command.

C:\>ping 192.168.1.2

Pinging 192.168.1.2 with 32 bytes of data:

Request timed out.
Request timed out.
Request timed out.
Request timed out.

Ping statistics for 192.168.1.2:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),

C:\>ping 192.168.1.3

Pinging 192.168.1.3 with 32 bytes of data:

Reply from 192.168.1.3: bytes=32 time<1ms TTL=128
Reply from 192.168.1.3: bytes=32 time<1ms TTL=128
Reply from 192.168.1.3: bytes=32 time=1ms TTL=128
Reply from 192.168.1.3: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.1.3:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 1ms, Average = 0ms

C:\>|
```

In this experiment, a network was created using two switches and multiple PCs in Cisco Packet Tracer to understand VLAN configuration and trunking.

First, two switches (2960) and four PCs were placed in the topology. The switches were connected using FastEthernet 0/24 ports, and the PCs were connected to FastEthernet ports on each switch.

Two VLANs were created on both switches:
VLAN 10 (Team1) and VLAN 20 (Team2).

The configuration commands used were:

```
enable
configure terminal
vlan 10
name Team1
vlan 20
```


name Team2

Next, ports were assigned to VLANs:

On each switch,

FastEthernet 0/1 was assigned to VLAN 10

FastEthernet 0/2 was assigned to VLAN 20

Commands used:

```
interface fa0/1
```

```
switchport mode access
```

```
switchport access vlan 10
```

```
interface fa0/2
```

```
switchport mode access
```

```
switchport access vlan 20
```

After assigning VLANs, a trunk link was configured between the two switches to allow VLAN traffic to pass between them.

The trunk was configured on FastEthernet 0/24 using:

```
interface fa0/24
```

```
switchport mode trunk
```

```
switchport trunk allowed vlan 10,20
```

Then, IP addresses were assigned to the PCs:

PC0 – 192.168.1.1

PC1 – 192.168.1.2

PC2 – 192.168.1.3

PC3 – 192.168.1.4

All with subnet mask 255.255.255.0.

Connectivity was tested using the ping command.

It was observed that:

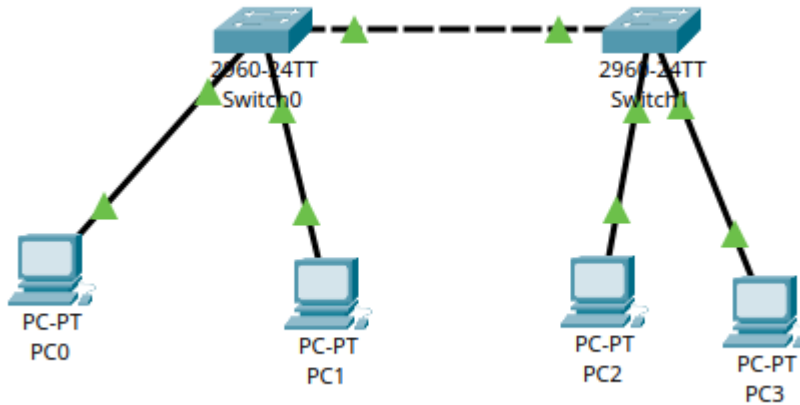
- Devices in the same VLAN (for example, VLAN 10 across switches) were able to communicate successfully.
- Devices in different VLANs (VLAN 10 and VLAN 20) were not able to communicate and showed request timed out.

This is because VLANs logically separate networks, and communication between different VLANs requires a Layer 3 device such as a router.

Thus, the experiment successfully demonstrated VLAN segmentation and trunking between switches.

6.Change the native VLAN on the trunk port. Test the VLAN mismatch and troubleshoot

Used same topology here:



%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/24, changed state to up
show interfaces status

Port	Name	Status	Vlan	Duplex	Speed	Type
Fa0/1		connected	10	a-full	a-100	10/100BaseTX
Fa0/2		connected	20	a-full	a-100	10/100BaseTX
Fa0/3		notconnect	1	auto	auto	10/100BaseTX
Fa0/4		notconnect	1	auto	auto	10/100BaseTX
Fa0/5		notconnect	1	auto	auto	10/100BaseTX
Fa0/6		notconnect	1	auto	auto	10/100BaseTX
Fa0/7		notconnect	1	auto	auto	10/100BaseTX
Fa0/8		notconnect	1	auto	auto	10/100BaseTX
Fa0/9		notconnect	1	auto	auto	10/100BaseTX
Fa0/10		notconnect	1	auto	auto	10/100BaseTX
Fa0/11		notconnect	1	auto	auto	10/100BaseTX
Fa0/12		notconnect	1	auto	auto	10/100BaseTX
Fa0/13		notconnect	1	auto	auto	10/100BaseTX
Fa0/14		notconnect	1	auto	auto	10/100BaseTX
Fa0/15		notconnect	1	auto	auto	10/100BaseTX
Fa0/16		notconnect	1	auto	auto	10/100BaseTX
Fa0/17		notconnect	1	auto	auto	10/100BaseTX
Fa0/18		notconnect	1	auto	auto	10/100BaseTX
Fa0/19		notconnect	1	auto	auto	10/100BaseTX
Fa0/20		notconnect	1	auto	auto	10/100BaseTX
Fa0/21		notconnect	1	auto	auto	10/100BaseTX
Fa0/22		notconnect	1	auto	auto	10/100BaseTX
Fa0/23		notconnect	1	auto	auto	10/100BaseTX
Fa0/24		connected	trunk	a-full	a-100	10/100BaseTX
Gig0/1		notconnect	1	auto	auto	10/100/1000BaseTX

--More--

All connections are good here:

```
Switch#show interfaces trunk
Port      Mode      Encapsulation  Status      Native vlan
Fa0/24    on        802.1q         trunking    1

Port      Vlans allowed on trunk
Fa0/24    10,20

Port      Vlans allowed and active in management domain
Fa0/24    10,20

Port      Vlans in spanning tree forwarding state and not pruned
Fa0/24    10,20
```

Here we change the native vlan to 2 in switch-0 which causes an warning:

```
Switch#enable
Switch#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Switch(config)#vlan 2
Switch(config-vlan)#name TEST
Switch(config-vlan)#exit
Switch(config)#int fa0/24
Switch(config-if)#switchport trunk native vlan 2
Switch(config-if)#
Switch#
%SYS-5-CONFIG_I: Configured from console by console

Switch#show interfaces trunk
Port      Mode      Encapsulation  Status      Native vlan
Fa0/24    on        802.1q         trunking    2

Port      Vlans allowed on trunk
Fa0/24    10,20

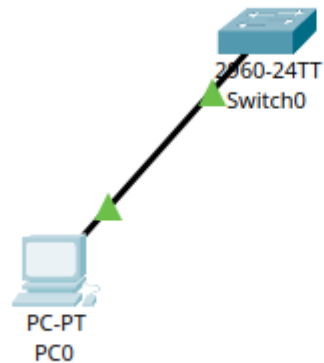
Port      Vlans allowed and active in management domain
Fa0/24    10,20

Port      Vlans in spanning tree forwarding state and not pruned
Fa0/24    10,20

Switch#
%CDP-4-NATIVE_VLAN_MISMATCH: Native VLAN mismatch discovered on FastEthernet0/24 (2), with
Switch FastEthernet0/24 (1).
```

- In this experiment, VLAN mismatch was created and analyzed using Cisco Packet Tracer.
- Initially, a trunk link was established between two switches with the default native VLAN set to VLAN 1 on both sides. Communication between devices was working correctly.
- To simulate a VLAN mismatch, the native VLAN on one switch was changed to VLAN 2 using the command:
 - switchport trunk native vlan 2
- After this configuration, a warning message indicating a native VLAN mismatch was observed.
- When connectivity was tested using the ping command, communication between devices failed.
- This is because native VLAN mismatch causes frames to be interpreted incorrectly between switches, leading to communication failure.
- To troubleshoot the issue, the trunk configuration was verified using the show interfaces trunk command.
- The problem was resolved by setting the native VLAN back to VLAN 1 on both switches:
 - switchport trunk native vlan 1
- After fixing the mismatch, communication between devices was restored successfully.
- This experiment demonstrates the importance of consistent native VLAN configuration in trunk links.

7. Configure a management VLAN and assign an IP address for remote access. Test SSH or Telnet access to switch



```
Switch#enable
Switch#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Switch(config)#vlan 99
Switch(config-vlan)#name MANAGEMENT
Switch(config-vlan)#exit
Switch(config)#int fa0/1
Switch(config-if)#switchport mode access
Switch(config-if)#switchport access vlan 99
Switch(config-if)#interface vlan 99
Switch(config-if)#
%LINK-5-CHANGED: Interface Vlan99, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface Vlan99, changed state to up

Switch(config-if)#ip address 192.168.1.10 255.255.255.0
Switch(config-if)#no shutdown
Switch(config-if)#exit
Switch(config)#ip default-gateway 192.168.1.1
Switch(config)#
```

Physical Config **Desktop** Programming Attributes

IP Configuration

Interface FastEthernet0

IP Configuration

☐ DHCP ☒ Static

IPv4 Address 192.168.1.2

Subnet Mask 255.255.255.0

Default Gateway 192.168.1.1

DNS Server 0.0.0.0

IPv6 Configuration

☐ Automatic ☒ Static

IPv6 Address /

Link Local Address FE80::20B:BEFF:FEEC:2A00

```
C:\>ping 192.168.1.10

Pinging 192.168.1.10 with 32 bytes of data:

Reply from 192.168.1.10: bytes=32 time<1ms TTL=255
Reply from 192.168.1.10: bytes=32 time<1ms TTL=255
Reply from 192.168.1.10: bytes=32 time<1ms TTL=255
Reply from 192.168.1.10: bytes=32 time<1ms TTL=255

Ping statistics for 192.168.1.10:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms
```

```
Switch#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Switch(config)#line vty 0 4
Switch(config-line)#password cisco
Switch(config-line)#login
Switch(config-line)#exit
Switch(config)#hostname Switch1
Switch1(config)#ip domain-name example.com
Switch1(config)#crypto key generate rsa
The name for the keys will be: Switch1.example.com
Choose the size of the key modulus in the range of 360 to 4096 for your
  General Purpose Keys. Choosing a key modulus greater than 512 may take
  a few minutes.

How many bits in the modulus [512]: 1024
% Generating 1024 bit RSA keys, keys will be non-exportable...[OK]

Switch1(config)#username admin password cisco
*Mar 1 0:46:27.424: %SSH-5-ENABLED: SSH 1.99 has been enabled
Switch1(config)#line vty 0 4
Switch1(config-line)#transport input ssh
Switch1(config-line)#login local
Switch1(config-line)#exit
Switch1(config)#
```

```
C:\>telnet 192.168.1.10
Trying 192.168.1.10 ...Open

User Access Verification

Password:
% Password: timeout expired!

[Connection to 192.168.1.10 closed by foreign host]
C:\>telnet 192.168.1.10
Trying 192.168.1.10 ...Open

User Access Verification

Password:
Switch1>
```

```
C:\>ssh -l admin 192.168.1.10
Password: |
```

In this experiment, a management VLAN was configured on a switch to enable remote access using Telnet and SSH protocols.

A simple topology consisting of one switch and one PC was created in Cisco Packet Tracer. The PC was connected to the switch using a FastEthernet interface.

First, a management VLAN (VLAN 99) was created on the switch using the following commands:

```
enable
configure terminal
vlan 99
name MANAGEMENT
```

Next, the PC-connected port was assigned to VLAN 99:
interface fa0/1


```
switchport mode access  
switchport access vlan 99
```

After creating the VLAN, an IP address was assigned to the switch using a virtual interface:

```
interface vlan 99  
ip address 192.168.1.10 255.255.255.0  
no shutdown
```

A default gateway was configured to allow communication with other networks:

```
ip default-gateway 192.168.1.1
```

The PC was configured with the following IP details:

```
IP Address: 192.168.1.2  
Subnet Mask: 255.255.255.0  
Default Gateway: 192.168.1.1
```

Connectivity between the PC and the switch was verified using the ping command.

For Telnet configuration, VTY lines were configured with password authentication:

```
line vty 0 4  
password cisco  
login
```

For SSH configuration, secure access was enabled by configuring hostname, domain name, and generating RSA keys:

```
hostname Switch1  
ip domain-name test.com  
crypto key generate rsa
```

A user account was created for SSH login:

```
username admin password 1234
```

The VTY lines were configured to allow only SSH access:

```
line vty 0 4  
login local  
transport input ssh
```

Finally, remote access was tested from the PC using:

```
telnet 192.168.1.10  
ssh -l admin 192.168.1.10
```

The switch successfully prompted for credentials and allowed login, confirming that remote access was configured correctly.

This experiment demonstrates how a management VLAN can be used to securely manage network devices using Telnet and SSH protocols.

8.You have a CISCO switch and a VoIP phone that need to be placed in a voice VLAN (VLAN 20). The data for the PC should remain in a separate VLAN (VLAN 10). Configure the switch port to support both voice and data traffic



```
Switch#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Switch(config)#vlan 10
Switch(config-vlan)#name DATA
Switch(config-vlan)#exit
Switch(config)#vlan 20
Switch(config-vlan)#name VOICE
Switch(config-vlan)#exit
Switch(config)#interface fa0/1
Switch(config-if)#switchport mode access
Switch(config-if)#switchport access vlan 10
Switch(config-if)#switchport voice vlan 20
Switch(config-if)#
```

```
Switch#show vlan brief
```

VLAN	Name	Status	Ports
1	default	active	Fa0/2, Fa0/3, Fa0/4, Fa0/5 Fa0/6, Fa0/7, Fa0/8, Fa0/9 Fa0/10, Fa0/11, Fa0/12, Fa0/13 Fa0/14, Fa0/15, Fa0/16, Fa0/17 Fa0/18, Fa0/19, Fa0/20, Fa0/21 Fa0/22, Fa0/23, Fa0/24, Gig0/1 Gig0/2
10	DATA	active	Fa0/1
20	VOICE	active	Fa0/1
1002	fddi-default	active	
1003	token-ring-default	active	
1004	fddinet-default	active	
1005	trnet-default	active	

```
Switch#
```

In this experiment, a Cisco switch was configured to support both voice and data traffic on a single port using Voice VLAN.

A topology consisting of a switch, an IP phone, and a PC was created. The PC was connected to the IP phone, and the phone was connected to the switch.

Two VLANs were created:
VLAN 10 for data traffic and VLAN 20 for voice traffic.

The switch port connected to the IP phone was configured using the following commands:

```
interface fa0/1
switchport mode access
switchport access vlan 10
switchport voice vlan 20
```

The command `switchport access vlan 10` assigns the PC's data traffic to VLAN 10, while `switchport voice vlan 20` assigns the IP phone's voice traffic to VLAN 20.

This allows both voice and data traffic to be transmitted through the same physical port while keeping them logically separated.

The IP phone automatically recognizes the voice VLAN and operates in VLAN 20, while the PC traffic remains in VLAN 10.

This configuration improves network performance and ensures proper prioritization of voice traffic.

9.You configured VLAN 10 and 20 on your switch and assigned ports to each VLAN, however, devices in VLAN 10 can't communicate with devices in VLAN 20.

Troubleshoot the issue.

In this scenario, devices in VLAN 10 are unable to communicate with devices in VLAN 20.

This is expected behavior because VLANs are designed to logically separate networks. Devices in different VLANs cannot communicate directly without Layer 3 routing.

To troubleshoot the issue, the following steps were performed:

1. Verified VLAN configuration using the `show vlan brief` command to ensure that ports were correctly assigned to VLAN 10 and VLAN 20.
2. Checked trunk configuration using `show interfaces trunk` to ensure that VLAN traffic is allowed between switches.
3. Verified IP addressing of devices to ensure they belong to the correct subnet.

4. Checked default gateway configuration for each VLAN.

The main issue identified is the absence of inter-VLAN routing. Since switches operate at Layer 2, they cannot route traffic between VLANs.

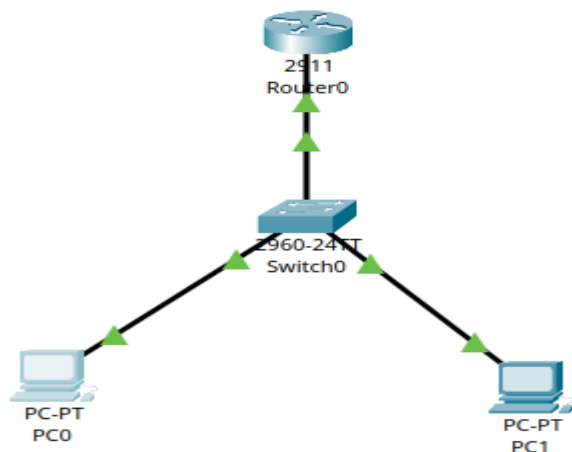
To resolve the issue, a router or Layer 3 switch must be used.

Router-on-a-stick configuration can be implemented by creating subinterfaces for each VLAN and assigning IP addresses.

After configuring inter-VLAN routing, devices in VLAN 10 and VLAN 20 can successfully communicate.

Thus, the issue is not a fault but a design characteristic of VLANs, and it can be resolved by enabling routing between VLANs.

10. Try inter-VLAN routing using Router



```
Switch>enable
Switch#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Switch(config)#vlan 10
Switch(config-vlan)#name DATA
Switch(config-vlan)#exit
Switch(config)#vlan 20
Switch(config-vlan)#name VOICE
Switch(config-vlan)#exit
Switch(config)#int fa0/1
Switch(config-if)#switchport mode access
Switch(config-if)#switchport access vlan 10
Switch(config-if)#int fa0/2
Switch(config-if)#switchport mode access
Switch(config-if)#switchport access vlan 20
Switch(config-if)#int fa0/24
Switch(config-if)#switchport mode trunk
Switch(config-if)#
```

```
Router>enable
Router#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#interface g0/0.10
Router(config-subif)#encapsulation dot1q 10
Router(config-subif)#ip address 192.168.10.1 255.255.255.0
Router(config-subif)#interface g0/0.20
Router(config-subif)#encapsulation dot1q 20
Router(config-subif)#ip address 192.168.20.1 255.255.255.0
Router(config-subif)#interface g0/0
      ^
% Invalid input detected at '^' marker.

Router(config-subif)#
Router#
%SYS-5-CONFIG_I: Configured from console by console
conf t
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#interface g0/0
      ^
% Invalid input detected at '^' marker.

Router(config)#interface g0/0
Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0, changed state to up

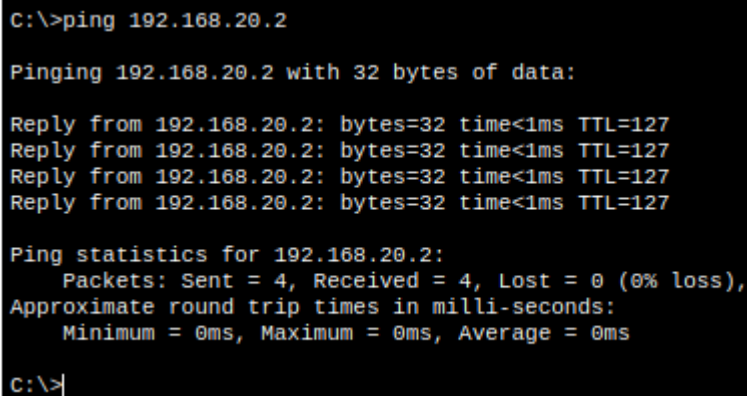
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up

%LINK-3-UPDOWN: Interface GigabitEthernet0/0.10, changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0.10, changed state to up

%LINK-3-UPDOWN: Interface GigabitEthernet0/0.20, changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0.20, changed state to up
|
```



```
C:\>ping 192.168.20.2

Pinging 192.168.20.2 with 32 bytes of data:

Reply from 192.168.20.2: bytes=32 time<1ms TTL=127
Reply from 192.168.20.2: bytes=32 time<1ms TTL=127
Reply from 192.168.20.2: bytes=32 time<1ms TTL=127
Reply from 192.168.20.2: bytes=32 time<1ms TTL=127

Ping statistics for 192.168.20.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\>|
```

In this experiment, inter-VLAN routing was implemented using a router (Router-on-a-Stick method).

A switch and a router were connected, and two VLANs (VLAN 10 and VLAN 20) were created on the switch. Devices were assigned to their respective VLANs.

The switch port connected to the router was configured as a trunk to carry multiple VLAN traffic.

On the router, subinterfaces were created for each VLAN. Each subinterface was assigned an IP address and configured with encapsulation dot1Q corresponding to the VLAN ID.

```
interface g0/0.10
encapsulation dot1Q 10
ip address 192.168.10.1 255.255.255.0
```

```
interface g0/0.20
encapsulation dot1Q 20
ip address 192.168.20.1 255.255.255.0
```

PCs were configured with IP addresses and default gateways pointing to the router.

Initially, devices in different VLANs could not communicate. After configuring inter-VLAN routing, successful communication was achieved using ping.

This demonstrates that routers enable communication between different VLANs by routing traffic between them.

11. Implement ACL to restrict traffic based on source and destination ports. Test rules by simulating legitimate and unauthorized traffic

Same topo from the above:

Changed the access alone here in router:

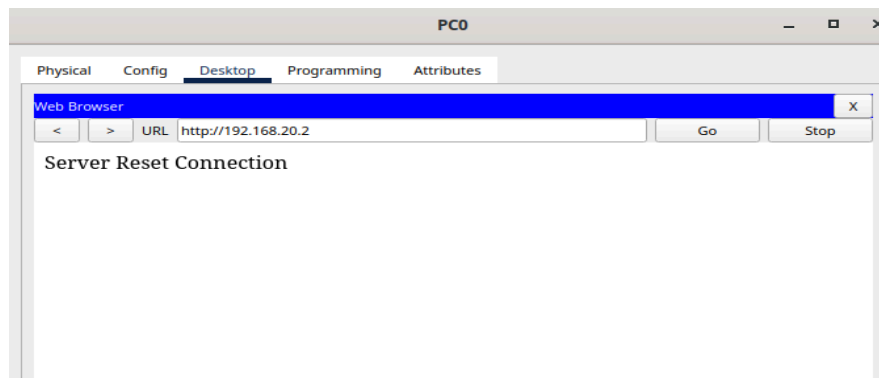
```
Router(config)#access-list 100 deny tcp any any eq 80
Router(config)#access-list 100 permit ip any any
Router(config)#interface g0/0
Router(config-if)#ip access-group 100 in
Router(config-if)#
```

```
C:\>ping 192.168.20.2

Pinging 192.168.20.2 with 32 bytes of data:

Reply from 192.168.20.2: bytes=32 time<1ms TTL=127
Reply from 192.168.20.2: bytes=32 time<1ms TTL=127
Reply from 192.168.20.2: bytes=32 time<1ms TTL=127
Reply from 192.168.20.2: bytes=32 time<1ms TTL=127

Ping statistics for 192.168.20.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms
```



The ACL was tested by sending both allowed and restricted traffic.

ICMP traffic (ping) between devices was successful, indicating that normal communication is allowed.

HTTP traffic was tested using a web browser by accessing <http://192.168.20.2>. The connection failed and showed "Server Reset Connection", confirming that HTTP traffic (port 80) is blocked by the ACL.

Thus, the ACL successfully restricts unauthorized traffic while allowing legitimate communication.

12.12) Configure a standard ACL on a router to allow traffic from a particular specific range of IP. Test connectivity and verify that the ACL is working as intended.

Same topo as above here too:

```
Router#enable
Router#
% Unknown command or computer name, or unable to find computer address

Router#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#access-list 10 permit 192.168.10.0 0.0.0.255
Router(config)#access-list 10 deny any
Router(config)#
```

```
C:\>ping 192.168.10.2

Pinging 192.168.10.2 with 32 bytes of data:

Reply from 192.168.10.2: bytes=32 time=4ms TTL=128
Reply from 192.168.10.2: bytes=32 time=1ms TTL=128
Reply from 192.168.10.2: bytes=32 time=2ms TTL=128
Reply from 192.168.10.2: bytes=32 time=1ms TTL=128

Ping statistics for 192.168.10.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 1ms, Maximum = 4ms, Average = 2ms

C:\>ping 192.168.20.2

Pinging 192.168.20.2 with 32 bytes of data:

Request timed out.
Request timed out.
Request timed out.
Request timed out.

Ping statistics for 192.168.20.2:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),
```

In this experiment, a standard ACL was configured to allow traffic only from a specific IP range.

The ACL was created to permit traffic from 192.168.10.0/24 and deny all other traffic.


```
access-list 10 permit 192.168.10.0 0.0.0.255  
access-list 10 deny any
```

Initially, both networks were able to communicate, indicating incorrect ACL placement. The issue was resolved by applying the ACL on the correct subinterface (g0/0.20) in the inbound direction.

After applying the ACL correctly, testing was performed using ping.

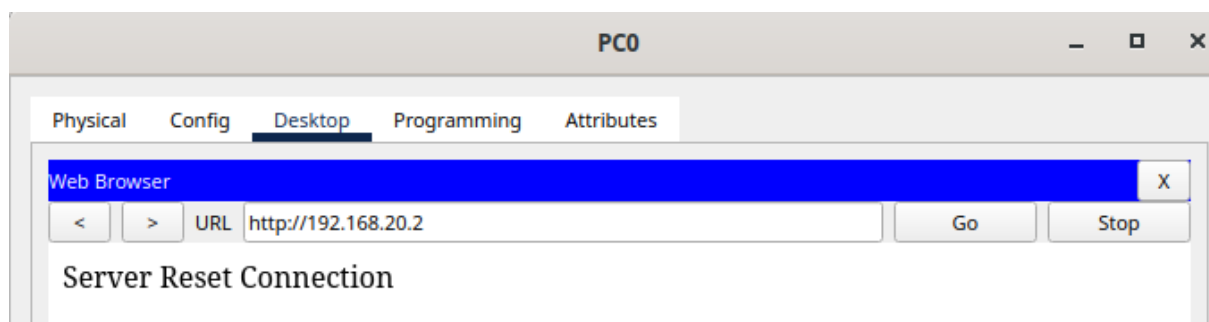
Devices from the allowed network (192.168.10.0/24) were able to communicate successfully. Devices from other networks were unable to communicate, as shown by request timeouts.

Thus, the ACL successfully restricted traffic based on the source IP address and worked as intended.

13.Create an extended ACL to block specific applications, such as HTTP or FTP traffic. Test the ACL rule by attempting to access the blocked services

```
Router#enable  
Router#conf t  
Enter configuration commands, one per line. End with CNTL/Z.  
Router(config)#access-list 100 deny tcp any any eq 80  
Router(config)#access-list 100 deny tcp any any eq 21  
Router(config)#access-list 100 permit ip any any  
Router(config)#interface g0/0.20  
Router(config-subif)#ip access-group 100 in  
Router(config-subif)#
```

```
C:\>ping 192.168.20.2  
  
Pinging 192.168.20.2 with 32 bytes of data:  
  
Reply from 192.168.20.2: bytes=32 time=6ms TTL=127  
Reply from 192.168.20.2: bytes=32 time<1ms TTL=127  
Reply from 192.168.20.2: bytes=32 time<1ms TTL=127  
Reply from 192.168.20.2: bytes=32 time=2ms TTL=127  
  
Ping statistics for 192.168.20.2:  
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),  
Approximate round trip times in milli-seconds:  
    Minimum = 0ms, Maximum = 6ms, Average = 2ms
```



```
C:\>ftp 192.168.20.2
Trying to connect...192.168.20.2

%Error ftp://192.168.20.2/ (Ftp peer reset)

(Disconnecting from ftp server)
```

In this experiment, an extended Access Control List (ACL) was configured to block specific application traffic such as HTTP and FTP.

The following ACL rules were created:

```
access-list 100 deny tcp any any eq 80
access-list 100 deny tcp any any eq 21
access-list 100 permit ip any any
```

The ACL was applied on the router subinterface (g0/0.20) in the inbound direction.

To test the configuration, ICMP traffic was verified using ping, which was successful, indicating allowed traffic.

HTTP access was tested using a web browser and resulted in a failed connection, confirming that port 80 was blocked.

FTP access was also tested using the ftp command and failed, confirming that port 21 was blocked.

Thus, the extended ACL successfully restricted specific application traffic while allowing other types of communication.